Diseño y Arquitectura de Software

Sistema de Banca por Internet - Entidad BP

Preparado por: Alejandro Llano
Fecha: 23 oct 2026

Índice de Contenido

1. **Funcionalidad**
2. **Validación Documental (OCR y Biometría)**
3. **Autenticación y Gestión de Identidad (IdP)**
4. **Tecnología Frontend (App móvil y web)**
5. **Tecnología Backend**
6. **Base de datos**
7. **Infraestructura cloud**
8. **Marcos Normativos y Legales**
9. **Diagramas**

Diseño y arquitectura para el sistema de Banca por internet para la entidad BP

Funcionalidad

Historial de Transacciones
Información a mostrar: Tipo de transferencia, nombre del beneficiario, montos y saldos.

Filtros disponibles: fecha, cuentas involucradas y entidad financiera.

Criterios de organización: Por fechas y montos.

Acciones:

- Ver estado de cuenta.
- Configuración de cuenta (definición de límites, asignación de nombre personalizado, bloqueo de cuenta).
- Verificar transferencias.

Módulo de Transferencias
Selección de destinatarios: Búsqueda de contactos registrados o creación/registro de nuevos contactos para la selección del beneficiario.

Definición de tipo de flujo: Diferenciación operativa entre transferencias directas e interbancarias.

Ingreso de datos: Especificar el monto a transferir, verificar saldo disponible o seleccionar la cuenta de origen, ingresar un motivo.

En caso de transferencias interbancarias, desplegar claramente el costo de la comisión por transacción.

Segunda verificación: Mostrar una pantalla de confirmación con el detalle completo de la transacción antes de procesar.

Procesamiento: Pantalla de carga (loading page) que espera internamente la respuesta de la API.

Reglas de Negocio, Integridad y Resiliencia en Backend
Validación transaccional: A nivel de base de datos para garantizar que, ante cualquier fallo, no se registren datos incompletos que alteren el saldo del cliente.

Mecanismos de reversión (Rollback): En caso de errores por fallos de infraestructura o código, el proceso debe reversarse completamente.

Gestión de la experiencia y seguridad: Mostrar mensajes claros al cliente para conservar su confianza en el sistema y prevenir transacciones duplicadas por reintentos indebidos o inconsistencias de datos.

Alta disponibilidad: Uso obligatorio de adaptadores asíncronos para garantizar la disponibilidad del servicio.

Seguridad e Infraestructura (Networking & APIs)
Cifrado en tránsito: Comunicación Front-End a Back-End correctamente cifrada bajo estándares TLS.

Protección perimetral y red: Implementación de Cloud Armor y VPCs privadas.

Integración con externos (APIs interbancarias): Configuración de listas blancas de IPs (whitelisting), túneles VPN para tráfico privado y esquemas de autenticación segura.

Módulo de Pago de Servicios
Catálogo de servicios: Listado interactivo de los servicios integrados al sistema BP disponibles para pago.

Conectividad con terceros: Configuración de conectividad segura con los sistemas externos de los proveedores de servicios, canalizando idealmente todo el tráfico a través de VPN.

Estándares de seguridad: Aplicación estricta de los mismos mecanismos de seguridad, validación transaccional y resiliencia implementados en el módulo de transferencias.

Validación Documental (OCR y Biometría)

Captura y extracción

Captura mediante cámara del documento de identidad y extracción automática de datos mediante OCR.

Validación de autenticidad

Uso de AWS Rekognition para validar la autenticidad del documento mediante umbrales de reconocimiento.

Registro Facial y Prueba de Vida (Liveness)

Proceso guiado: Captura tipo selfie asistida para registrar diferentes ángulos del rostro y análisis de profundidad para la prevención de fraudes.

Estándares de seguridad y cumplimiento

- Implementación de protocolos internacionales como FIDO 2 para la prevención de ataques de phishing.
- Algoritmos de detección de prueba de vida (Liveness) en servidores bajo la norma internacional ISO/IEC 30107-3 para mitigar suplantaciones mediante fotos o vídeos.

Verificación Final y Protección de Datos

Confirmación OTP: Envío de código de seguridad temporal (OTP) vía SMS y correo electrónico al completar con éxito las validaciones biométricas.

Cumplimiento normativo: Tratamiento de datos sensibles ajustado estrictamente a la Ley de Protección de Datos Personales.

Seguridad Biométrica en Dispositivo

Almacenamiento local seguro: Los datos biométricos reales para el acceso diario permanecen cifrados localmente dentro del chip seguro del dispositivo (Secure Enclave / TEE).

Confirmación criptográfica: El banco nunca recibe ni almacena la biometría en sus servidores; únicamente valida el acceso mediante una confirmación criptográfica enviada por el sistema operativo del teléfono.

Autenticación y Gestión de Identidad (IdP)

Vías de Acceso a la App

Esquema principal: Acceso tradicional mediante usuario y contraseña.

Acceso biométrico: Habilitación de lectura de huella dactilar y reconocimiento facial en dispositivos compatibles.

Opciones disponibles: Usuario y contraseña, huella dactilar, reconocimiento facial.

Integración con Proveedores de Identidad Externos (IdP)

Delegación de autenticación: Basada en estándares abiertos OpenID Connect (OIDC) y OAuth 2.0 PKCE.

Plataformas de identidad: Integración interoperable con Google Cloud Identity Platform o Auth0.

Políticas dinámicas de acceso: Soporte para Step-up Authentication y Autenticación Multi-Factor (MFA) contextual adaptativa, evaluando el nivel de riesgo según el dispositivo y la ubicación.

Tecnología Frontend (App móvil y web)

Aplicación Móvil: Flutter

- Ofrece un rendimiento predecible y un control absoluto de la arquitectura, con una filosofía estructurada muy similar a la de Angular.

  • Código unificado: El principal beneficio para este proyecto es que permite mantener una única base de código para múltiples plataformas (Android e iOS).

  • Lenguaje Dart: Su lenguaje de programación ha demostrado una consistencia superior a JavaScript. Al contar con tipado estricto nativo, previene errores en producción y asegura un óptimo desempeño.

  • Rendimiento cercano al nativo: Consigue una fluidez superior gracias a que utiliza su propio motor gráfico para renderizar las interfaces, evitando intermediarios.

  • Productividad optimizada: Su función de Hot Reload (recarga en vivo) agiliza drásticamente los ciclos de desarrollo al permitir visualizar cambios en tiempo real.

Aplicación Web: Angular

Al tratarse de un desarrollo con módulos sensibles y orientados al sector bancario que probablemente será abordado por un equipo numeroso de programadores, se considera indispensable el uso de este framework.

• Solución corporativa y estructurada: Garantiza que todos los miembros del equipo sigan estrictamente la misma arquitectura, eliminando la anarquía en el código.

• Tipado estricto con TypeScript: Su integración nativa ofrece beneficios críticos como la detección temprana de errores en tiempo de compilación y una alta mantenibilidad a largo plazo.

• Escalabilidad modular: Dado que es un proyecto con alta probabilidad de expandirse hacia múltiples módulos complejos para procesos bancarios, Angular es la herramienta ideal por estar diseñada específicamente para este propósito.

Otras Tecnologías Frontend Consideradas Descartadas

ReactJS

Pese a ser una alternativa popular para aplicaciones grandes, presenta limitaciones arquitectónicas importantes para este caso de uso:

• Falta de arquitectura nativa: No proporciona una estructura base, lo que traslada al equipo responsabilidades adicionales de diseño y mantenimiento que pueden fragmentar el proyecto.

• Ecosistema fragmentado: Carece de herramientas integradas propias, obligando a depender excesivamente de librerías de terceros para funciones críticas.

• TypeScript opcional: Al no exigir TypeScript de forma nativa desde su configuración inicial, limita los beneficios de control y seguridad que sí ofrece Angular de manera obligatoria.

React Native

Su arquitectura introduce riesgos operativos y de rendimiento que comprometen la escala del proyecto:

• Cuello de botella en el rendimiento: Depende fuertemente del entorno nativo del dispositivo y requiere de un puente de comunicación (bridge) para traducir el código JavaScript, lo que ralentiza la aplicación bajo cargas pesadas.

• Inestabilidad visual: Las actualizaciones del sistema operativo (iOS/Android) pueden romper el diseño de la interfaz al depender de los componentes nativos del teléfono.

• Lenguaje menos robusto: TypeScript (sobre JavaScript) no está diseñado a nivel de compilación para asegurar el performance de la aplicación de la forma en que Dart lo hace nativamente en Flutter.

Desarrollo Nativo Independiente (Android y iOS)

Aunque ofrece el máximo acceso al hardware, representa un desafío logístico insostenible para el modelo de negocio actual:

• Alto costo operativo: Exige financiar y mantener dos bases de código y dos equipos especializados independientes.

• Duplicación de tiempos: Se duplican los esfuerzos de desarrollo, los procesos de pruebas (QA) y se corre el riesgo latente de sufrir divergencias en la lógica de negocio entre ambas plataformas.

• Eficiencia de desarrollo reducida: Los tiempos de compilación nativos ralentizan el flujo de trabajo en comparación con la velocidad superior que demuestra el Hot Reload de Flutter.

Tecnología Backend

Spring Boot / Java

Ofrece un rendimiento altamente predecible, robustez empresarial y un control absoluto de la arquitectura, ideal para sistemas críticos que requieren alta disponibilidad y transaccionalidad.

• Solución corporativa y estandarizada: Al igual que Angular en el frontend, Spring Boot proporciona un marco de trabajo estrictamente estructurado. Esto garantiza que equipos grandes sigan los mismos patrones de diseño (como inyección de dependencias y arquitectura limpia), eliminando la anarquía en el código.

• Seguridad y robustez nativa: Cuenta con el ecosistema de seguridad más maduro del mercado (Spring Security), indispensable para cumplir con las normativas estrictas del sector bancario (auditoría, cifrado, control de accesos y gestión de sesiones).

• Tipado estricto y concurrencia madura: Java ofrece un tipado estricto que previene errores críticos en tiempo de compilación. Su gestión de hilos y memoria está diseñada para soportar millones de transacciones simultáneas sin degradar el rendimiento.

• Escalabilidad modular y empresarial: Está pensado nativamente para proyectos masivos. Facilita la transición de una arquitectura monolítica modular hacia microservicios, permitiendo expandir los procesos bancarios en módulos independientes de forma segura.

Otras Tecnologías Backend Consideradas Descartadas

NodeJS / Express (JavaScript/TypeScript)

• Falta de arquitectura nativa: Express es minimalista y no proporciona una estructura base. Traslada al equipo responsabilidades adicionales de diseño, lo que en proyectos grandes suele derivar en códigos inconsistentes y difíciles de auditar.

• Procesamiento intensivo limitado: Al estar basado en un solo hilo (Single-Threaded Event Loop), NodeJS es excelente para operaciones de Entrada/Salida (I/O), pero puede convertirse en un cuello de botella si se requiere procesamiento matemático pesado o criptografía compleja en tiempo real.

Python / Django o FastAPI

• Tipado dinámico original: Aunque soporta type hinting, no tiene el tipado estricto nativo de Java a nivel de compilación, lo que incrementa el riesgo de que errores de tipo o lógica sutiles lleguen a los entornos de producción.

• Rendimiento transaccional inferior: En pruebas de estrés de alta concurrencia y procesamiento puro de datos, los entornos de ejecución de Python suelen mostrar una latencia superior y un mayor consumo de recursos en comparación con la Máquina Virtual de Java (JVM).

Base de datos

Patrones de persistencia y caching

- En una aplicación bancaria los usuarios consultan constantemente su saldo, revisan sus movimientos o buscan contactos para realizar transferencias, se debe evitar que cada consulta vaya directamente a la base de datos principal ya que el sistema podría volverse lento cuando muchos usuarios estén conectados al mismo tiempo. Para evitarlo se puede implementar Cache-Aside con Redis guardando temporalmente información de consulta frecuente como los datos de los beneficiarios o la información necesaria para mostrar el saldo.

  También se debería aplicar CQRS separando las operaciones que modifican información de aquellas que únicamente la consultan para lograr que las transferencias se procesen mediante el flujo transaccional mientras que las consultas de historiales pueden atenderse desde una estructura optimizada para lectura.

Auditoría inmutable y telemetría

En una aplicación bancaria cada transferencia debe dejar un registro que permita registrar las acciones del cliente. Esto es fundamental para investigar errores, detectar operaciones sospechosas y cumplir con los requisitos de auditoría.

Para procesar y distribuir esos eventos se pueden utilizar herramientas como Apache Kafka o Amazon SQS. La inmutabilidad requiere controles adicionales, como permisos restrictivos, almacenamiento protegido y mecanismos de integridad.

Sería ideal la implementación de OpenTelemetry para seguir el recorrido de una solicitud a través de los distintos servicios. Por ejemplo, si un cliente realiza una transferencia y ésta tarda demasiado, un identificador permite investigar si el retraso ocurrió al validar el saldo, comunicarse con el banco receptor o registrar la operación. Esto facilita encontrar problemas sin tener que revisar manualmente cada servicio por separado.

API Gateway

Las solicitudes pueden provenir de diferentes lugares como la aplicación móvil, la página web o incluso herramientas externas como vimos anteriormente, para ello en lugar de permitir que cada uno se comunique directamente con todos los servicios internos, se puede utilizar un API Gateway como punto de entrada centralizado.

Este componente puede validar aspectos como la autenticación, aplicar límites de solicitudes para prevenir abusos y dirigir cada petición al servicio correspondiente.

Infraestructura cloud

Capa de Frontend (SPA)

Amazon S3 Bucket configurado en modo privado para alojar los artefactos estáticos compilados (index.html, bundles JS, CSS, imágenes).

Amazon CloudFront para la red de entrega de contenido (CDN) con soporte HTTPS global.

Origin Access Control para restringir el acceso al bucket S3 para que solo CloudFront pueda leer su contenido impidiendo acceso público directo.

AWS Certificate Manager para provisionar los certificados SSL/TLS asociados al dominio.

Capa de Red y Backend

Subredes públicas que alojan el balanceador de cargas a interfaces de salida NAT Gateway.

Subredes Privadas que alojen las tareas del backend sin acceso público directo. Subredes de Datos que alojen la base de datos sin ruta hacia internet.

AWS Fargate mediante Amazon ECS para la ejecución serverless de contenedores Docker del backend eliminando la administración de instancias EC2, manejo de autoscaling automático.

Amazon RDS para la base de datos relacional SQLServer

AWS Secrets Manager para la gestión segura de variables de entorno, claves de API y credenciales de BD.

Pipeline CI/CD (GitHub Actions)

La opción estándar GitHub Actions conectado a AWS mediante OIDC para evitar almacenar llaves fijas

Microservicios propuestos

Notificaciones: Procesa y despacha colas de envío de correos, SMS y notificaciones Push sin bloquear los servicios transaccionales principales.

Sesión y Antifraude: Analiza en tiempo real el comportamiento del usuario, ubicación geográfica, nivel de riesgo del dispositivo y estado de la cuenta previo a autorizar movimientos delicados.

Seguridad, Monitoreo y Observabilidad

Alta Disponibilidad (HA) y Auto-healing
Microservicios orquestados en Amazon EKS (Kubernetes) distribuidos en al menos 3 Zonas de Disponibilidad (Multi-AZ). Auto-scaling horizontal de Pods (HPA) y Auto-healing automático mediante Liveness/Readiness probes que reemplazan contenedores anómalos sin intervención humana.

Recuperación ante Desastres (DR)

Estrategia Warm Standby Active-Passive entre regiones AWS. Réplica continua asíncrona de datos con RPO < 1 min y RTO < 15 min, asegurando continuidad de negocio ante contingencias regionales completas.

Cifrado AES-256 en reposo y TLS 1.3 en tránsito. Inspección por AWS WAF y Shield Advanced. Monitoreo centralizado con Prometheus, Grafana y CloudWatch bajo el esquema de métricas RED (Rate, Errors, Duration).

Optimización de Costos y Excelencia Operativa

Uso de instancias Karpenter / Spot para cargas no críticas de procesamiento por lotes, autoscaling responsivo para evitar sobreaprovisionamiento y políticas de ciclo de vida en S3/Glacier para datos históricos de auditoría.

- Integración con servicios externos
    - Interfaces con proveedores KYC (Jumio/Rekognition)
    - Inicio de sesión Amazon Cognito
    - Servicios de mensajería (FCM/APNs/SES/Twilio) desacoplados a través de buses de eventos asíncronos y patrones de resiliencia (Circuit Breaker con Resilience4j)
    - AWS API Gateway para no exponer la complejidad interna de la red bancaria hacia Internet, encargándose de validar la seguridad (tokens OAuth 2.0), aplicar políticas de límite de peticiones (Throttling contra ataques de denegación de servicio)

Marcos Normativos y Legales

- El diseño propuesto contempla el cumplimiento de normativas financieras:
    1. **Protección de Datos Personales:** Aplicación estricta de cifrado fuerte en reposo (AES-256) en bases de datos y en tránsito (TLS 1.3) para todas las conexiones. En entornos de pruebas o desarrollo se aplican técnicas automáticas de enmascaramiento y anonimización de datos reales.
    2. **Cumplimiento PCI-DSS:** Para el manejo de datos de tarjetas de débito o crédito, los componentes receptores se aíslan en una zona de red de máxima seguridad (CDE - Cardholder Data Environment), prohibiendo explícitamente el almacenamiento de códigos CVV o PINs.

Diagramas

Diagrama 1: Contexto (C4 - Nivel 1)

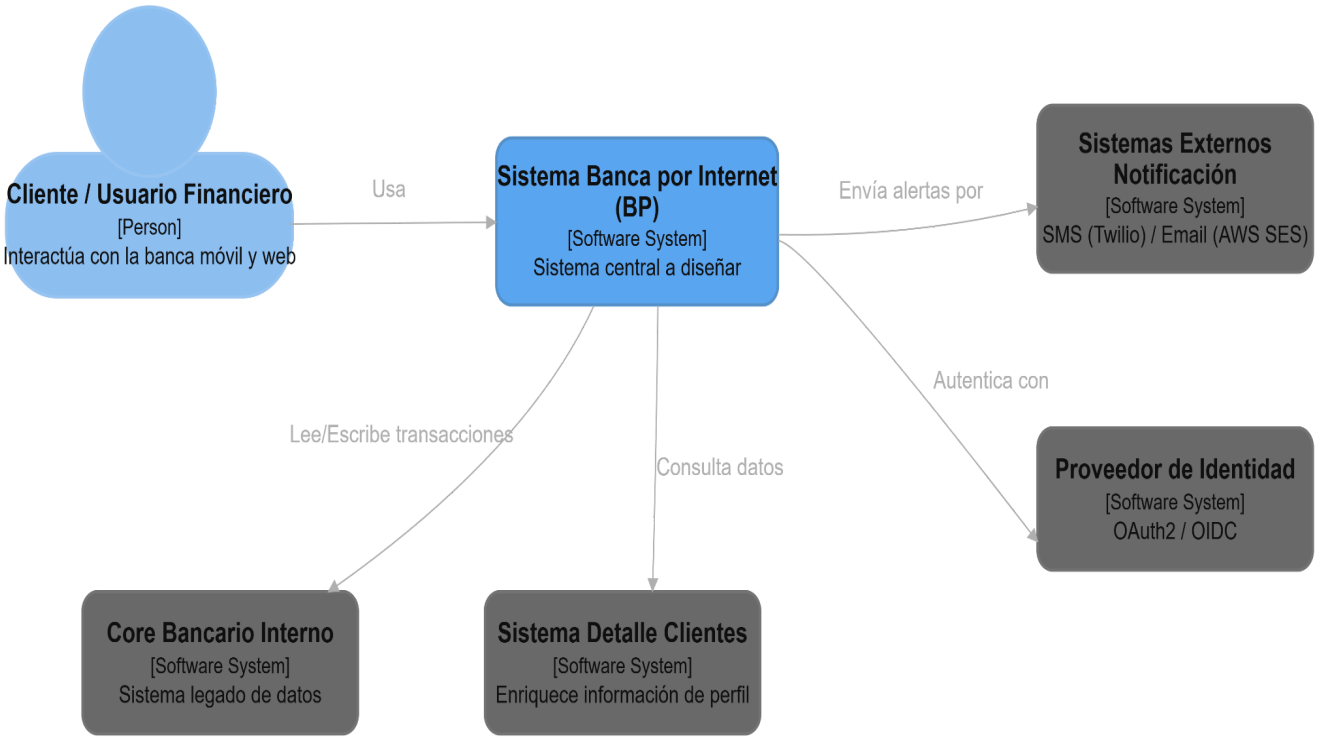


Diagrama 2: Contenedores (C4 - Nivel 2)

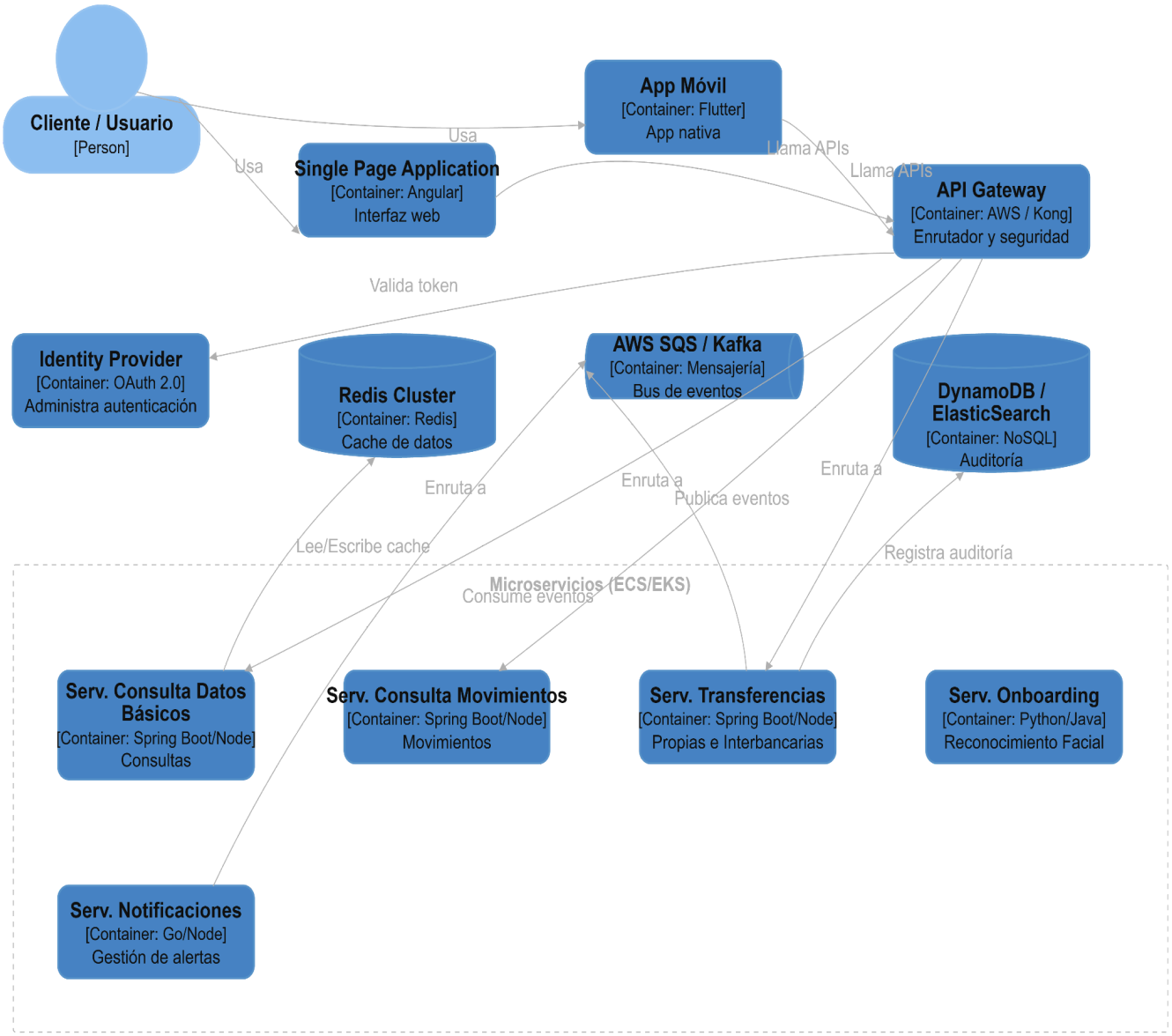


Diagrama 3: Componentes (C4 - Nivel 3)

